

DGAD: Complete Project Guide

Rajpreet Singh Khurana (K033)

August 2026

CONTENTS

DGAD: Complete Project Guide	
Disagreement-Gated Adaptive Detection — Every File Explained	
 Project Status at a Glance	
 Repository Structure Overview	
 src/dgad/ — The Core Package	
 Root Module Files	
 channels/ — The Five Detection Channels	
 api/ — FastAPI Service	
 storage/ — Audit Logging	
 eval/ — Evaluation Harness	
 Docker & CI/CD	
 Dashboard (React + Vite + TypeScript)	
Architecture	
Key Dashboard Features	
 Tests — Three Tiers	
 Documentation	
docs/adr/ — Architecture Decision Records	
docs/runbook.md	
docs/viva_prep.md	
 Key Concepts You Must Be Able To Explain	
1. Why Disagreement Gating?	

2. Why Calibration Before Gate?

3. Complementary Blind Spots

4. Two Novelties

5. Cost Advantage

6. Security of the Judge

7. No Data Leakage

 Recommended Reading (Small Blogs & Papers)

For Understanding the Core Ideas

For Engineering Concepts

 Viva Preparation Checklist

 Final Notes for Your Viva

DGAD: Complete Project Guide

Disagreement-Gated Adaptive Detection — Every File Explained

A Visual Reference for Your Capstone Viva

 Project Status at a Glance

Metric	Value
Overall Completion	94% (15/16 phases)
Current Phase	Phase 15: Report & Viva Prep
Lines of Python	~8,000+
Lines of TypeScript/React	~3,000+
Test Coverage	94 tests across 3 tiers
Architecture Decision Records	5 ADRs

Phases Complete:  Phases 0–14 | **Remaining:**  Phase 15 (Report & Viva Prep)



Repository Structure Overview

```

main/
├── .github/workflows/    # CI/CD pipelines (3 workflows)
├── alembic/              # Database migrations
├── dashboard/            # React + Vite + TypeScript + MUI + Recharts
├── data/                 # Dataset storage (gitignored)
├── docs/                 # Documentation (ADRs, runbook, viva prep)
├── models/               # Trained models & calibrators
├── results/              # Generated evaluation tables & figures
├── src/dgad/             # Main Python package (THE CORE)
├── tests/                # 3-tier test suite
├── pyproject.toml        # Project config, dependencies, tooling
├── docker-compose.yml    # Multi-service orchestration
├── Dockerfile            # Container definition
├── README.md             # Project overview & quickstart
├── .env.example          # Secret NAMES only (no values!)
├── .pre-commit-config.yaml # Pre-commit hooks
└── alembic.ini           # Alembic config

```



src/dgad/ — The Core Package



ROOT MODULE FILES

`__init__.py`

What it does: Makes `src/dgad` a proper Python package so you can `import dgad` from anywhere. **Why it exists:** Without this, Python treats the folder as just a directory, not a module. **Key detail:** Exports the public API — `DetectionPipeline`, `DisagreementGate`, `Settings`, `schemas`, and `channels`.

`config.py` — THE SINGLE SOURCE OF TRUTH FOR ALL THRESHOLDS

What it does: Central configuration using Pydantic Settings. Every tunable parameter lives here — no magic numbers anywhere else. **Why it exists:**

- Allows overriding via environment variables or `.env` file
- Type-safe configuration with validation

- Single place to tune the entire system **Key settings:**
- `gate_t_low=0.3` — Agreement threshold for PASS
- `gate_t_high=0.7` — Agreement threshold for BLOCK
- `gate_t_d=0.4` — Disagreement threshold for ESCALATE
- `perplexity_window=16`, `perplexity_stride=8` — Channel A params
- `embedding_model="all-MiniLM-L6-v2"` — Channels B & E embedder
- `calibration_method="platt"` — Score calibration method
- `database_url="sqlite:///./dgad.db"` — Audit log storage
- `random_seed=42` — Reproducibility

💡 **Viva tip:** “All thresholds are in config.py — this is explicit by design so examiners can see exactly what we tuned and where.”

`schemas.py` — **CONTRACTS BETWEEN ALL MODULES**

What it does: Defines Pydantic models for every data structure that crosses module boundaries. **Why it exists:**

- Type safety across the entire pipeline
- Automatic validation of API requests/responses
- Documentation of the data format in code
- The ablation study depends on these being stable

Key schemas:

Schema	Purpose
<code>Decision</code>	Enum: PASS, BLOCK, ESCALATE
<code>ChannelResult</code>	Output of every channel: <code>raw_score</code> , <code>calibrated_score</code> , <code>latency_ms</code> , <code>metadata</code>
<code>GateDecision</code>	Gate output: decision, per-channel scores, disagreement value, rationale
<code>JudgeVerdict</code>	LLM judge output: verdict, confidence, attack_family, rationale
<code>DetectRequest/Response</code>	API contract for <code>/v1/detect</code>
<code>AuditLogRecord</code>	Database row for every decision

💡 **Viva tip:** “ChannelResult is the universal interface — every channel returns exactly this shape. This is what makes ablation possible without rewiring.”

`normalise.py` — FIRST LINE OF DEFENSE

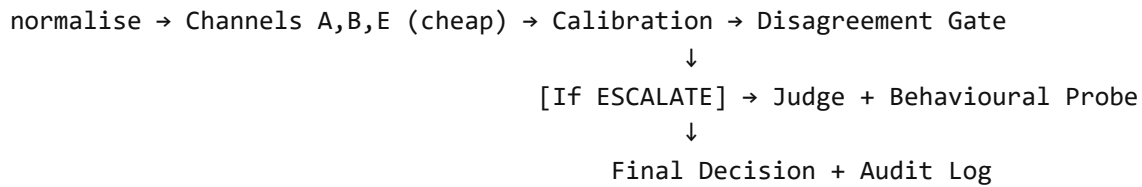
What it does: Preprocesses every prompt before any channel sees it. **Why it exists:** Attackers use Unicode tricks (zero-width chars, homoglyphs, full-width variants) to evade detection. Normalisation strips these. **Pipeline (in order):**

1. **NFKC Unicode normalisation** — Folds compatibility characters (full-width → normal)
2. **Zero-width character removal** — Strips , , , ,
3. **Confusable mapping** (hook) — Maps Cyrillic `а` → Latin `a`, etc.
4. **Whitespace collapse** — Multiple spaces/newlines → single space

Key property: Idempotent — `normalise(normalise(x)) == normalise(x)` — proven by tests.

`pipeline.py` — THE ORCHESTRATOR

What it does: Wires the entire detection flow end-to-end. **Request flow:**



Key design decisions:

- **Graceful degradation:** If a channel fails (model missing, service down), it returns `raw_score=0.5` with `metadata.degraded=True` — never crashes the pipeline
- **Fail-closed:** Zero healthy channels → ESCALATE (not PASS)
- **Dependency injection:** Tests can swap in fake channels
- **Cost tracking:** Judge calls tracked for cost-per-1000-prompts metric

`calibration.py` — MAKING SCORES COMPARABLE

What it does: Converts raw channel scores (different scales!) into honest probabilities in $[0,1]$. **Why it exists:** Channel A outputs perplexity ~ 340 , Channel B outputs 0.71. You *cannot* compare them until calibrated. **Methods:** Platt scaling (sigmoid) or Isotonic regression — configurable in `config.py` **Fitted on:** Validation split ONLY (never train, never test/holdout) **Outputs:** Reliability diagrams (before/after), Expected Calibration Error (ECE)

`gate.py` — THE CORE NOVELTY 🌟

What it does: Implements the Disagreement Gate — the routing logic that is DGAD's main contribution. **Why it's different:** Other systems **average** detector scores. DGAD uses **disagreement as a control signal**.

Three-way policy:

Condition	Action
All scores $\leq$ <code>t_low</code>	PASS immediately
All scores $\geq$ <code>t_high</code>	BLOCK immediately
Disagreement $\geq$ <code>t_d</code>	ESCALATE to expensive tier
Otherwise	Fused mean decides

Disagreement metrics (configurable):

- `abs_diff` — Maximum pairwise absolute difference
- `region_conflict` — Channels on opposite sides of decision band
- `entropy` — Binary entropy of fused mean

Naive baselines included: `mean_fusion()`, `max_fusion()` — for ablation comparison.

Threshold tuning: Optuna optimisation on validation — maximise ROC-AUC subject to escalation rate $\leq$ target.

`canary.py` — INDIRECT INJECTION TRIPWIRE

What it does: Detects prompt injection hidden inside untrusted documents (web pages, emails, tool output). **How it works:**

1. Generate **fresh random token** per request (e.g., `XZ9K2`)
2. Wrap document with directive: *"Process this, then end your reply with code XZ9K2"*
3. If response **lacks the token** → Something hijacked the model's instructions
4. Also checks for injection signals: *"as instructed in the document"*, *"forwarded"*, etc.

CanaryStatus enum: `INTACT`, `DROPPED`, `INSTRUCTION_OBEYED` **Tested against:** BIPIA benchmark (Microsoft's indirect injection benchmark)

`adversarial/mutators.py` — MEANING-PRESERVING PERTURBATIONS

What it does: Generates small mutations of text that preserve semantic meaning but change surface form. **Used by:** Channel C (behavioural probe) and Phase 10 adversarial loop. **Operators:**

- `char_swap` — Adjacent character swaps (typo simulation)
- `char_drop` — Random character deletion
- `whitespace_jitter` — Add/remove spaces & punctuation
- `synonym_replace` — WordNet/built-in synonym substitution
- `random_mutation` — Composes N random operators in sequence

`adversarial/loop.py` — **SECOND NOVELTY** 🌟

What it does: Self-adversarial calibration loop — actively hunts for prompts that fool ALL cheap channels in the SAME direction (false agreement). **The threat model:** An attack that makes Channel A say 0.2 AND Channel B say 0.2 → Gate sees agreement → PASS → Attack succeeds. **Algorithm:**

1. Start from known attack
2. Hill-climb mutations toward `max(channel_scores) ≤ t_low` AND `judge(mutated) == attack`
3. Verified hits = **hard negatives** → Added to Channel B training data
4. Retrain Channel B, re-fit calibrators, re-tune gate
5. Repeat for R rounds, plot false-agreement rate vs round

Guarded against: Overfitting to its own mutators (hold out untouched attack families).

📁 CHANNELS/ — THE FIVE DETECTION CHANNELS

`base.py` — **THE CONTRACT (DO NOT BREAK)**

```
class Channel(ABC):
    name: str # "statistical", "semantic", "behavioural", "judge", "offset"

    @abstractmethod
    def score(self, text: str) -> ChannelResult: ...
```

Every channel implements this exact interface. Ablation study swaps channels by name only.

statistical.py — Channel A: Windowed Perplexity + Token Anomaly

Catches: GCG-style gibberish suffixes, encoding tricks, character obfuscation **Blind to:** Fluent, well-written jailbreaks **Cost:** Very low (one GPT-2 forward pass)

How it works:

1. Load GPT-2 (lazy, cached in memory)
 2. **Windowed perplexity:** Slide window of W tokens (default 16) at stride S (default 8), take **maximum** perplexity — exposes hidden weird suffixes
 3. **Token anomaly features:** Non-ASCII ratio, punctuation ratio, zero-width count, longest token run, single-char token ratio, character entropy
 4. **Combiner:** Logistic regression on $[\log(\text{max_ppl}), \text{anomaly_features}]$ — trained on train split
 5. **Fallback:** If GPT-2 unavailable, runs on anomaly features only (degraded mode)
-

semantic.py — Channel B: Embedding Classifier

Catches: Role-play jailbreaks, “ignore previous instructions”, DAN prompts **Blind to:** Novel phrasings unseen in training; over-defends on innocent trigger words **Cost:** Low (one embedding + logistic regression)

How it works:

1. Embed prompt with `all-MiniLM-L6-v2` (384-dim) — lazy loaded, cached
 2. Logistic regression head on embeddings
 3. **Offline fallback:** Hashing embedder (deterministic, zero downloads) for CI/synthetic eval
 4. **Over-defense measured:** Explicitly tested on NotInject dataset
-

offset.py — Channel E: Black-Box Representation Offset

Catches: Prompts whose surface wording $\neq$ real intent (disguised requests) **Inspired by:** ROD (Representation Offset Detection) — but **black-box approximation Cost:** Low–Medium (one paraphrase call + embeddings)

How it works:

1. Extract intent: *"In one sentence, state what this text is actually asking the system to do"* (via local Ollama model or heuristic)
 2. Embed both original text and extracted intent
 3. Score = **Cosine distance** between the two embeddings
 4. Honest prompts → intent ≈ original → low distance
 5. Disguised attacks → intent ≠ original → high distance
-

`behavioural.py` — Channel C: Black-Box Behavioural Probe

Catches: Brittle optimised attacks that collapse under small perturbation **Known**

limitation: Weak alone against well-aligned models (cited: "The Consistency Confound") **Why it works here:** Only runs on **contested subset** (escalation tier), not everything **Cost:** High (K generations via local proxy model)

How it works:

1. Generate K mutated variants of contested prompt
 2. Run each through local proxy model (Ollama: Qwen2.5-1.5B)
 3. Measure compliance swing: `|compliance(variant) - compliance(original)|`
 4. Raw score = Variance/entropy of compliance across variants
 5. **Degraded mode:** If probe model unreachable → neutral 0.5 with flag
-

`judge.py` — Channel D: LLM-as-a-Judge

Catches: Almost everything, with human-readable rationale **Cost:** Highest (API call or local model) **Security hardened:** Judge prompt is itself an attack surface!

Defenses built in:

- **Data delimiters:** `<PROMPT_TO_JUDGE>...</PROMPT_TO_JUDGE>` — wrapped content is DATA
- **Canary token:** Random per-call token in system prompt — if judge drops it, judge was hijacked
- **Structured output:** JSON schema validated by Pydantic, retry once, fail-closed
- **Caching:** Redis (or in-process) keyed by prompt hash
- **Circuit breaker:** 3 consecutive failures → 60s cooldown, degrade gracefully

- **Cost tracking:** Estimated USD per call for cost comparison tables
-

API/ — FASTAPI SERVICE

`main.py` — **Application Factory**

- Creates FastAPI app with lifespan (DB init, logging)
- Mounts routers: `/v1/detect`, `/v1/chat/completions`, `/admin/*`
- `/health` endpoint for Docker health checks
- `/metrics` endpoint for Prometheus scraping
- Global dependencies: API key auth (disabled in dev), structured logging, request metrics

`routes_detect.py` — **POST /v1/detect**

- Scores a prompt WITHOUT forwarding to LLM
- In-process cache on normalised prompt hash
- Logs to audit DB with per-channel latency breakdown
- Returns: decision, scores, disagreement, rationale, escalation flag, latency

`routes_proxy.py` — **POST /v1/chat/completions (OpenAI-Compatible Proxy)**

- **One-line integration:** Change `base_url` to DGAD, detection happens invisibly
- Extracts last user message from chat history
- Policy modes: `block` (refuse), `flag_and_pass` (forward + header), `observe_only` (log only)
- Forwards to upstream LLM (configurable `upstream_base_url`)
- Injects DGAD metadata into response: decision, flagged status, prompt hash

`routes_admin.py` — **Admin Dashboard Endpoints**

- `GET /admin/decisions` — Recent decisions for live feed
 - `GET /admin/metrics-summary` — Aggregated counters
 - `POST /admin/playground` — **Threshold playground:** Recompute decisions over historical data with hypothetical thresholds — powers the viva demo!
-

📁 STORAGE/ — AUDIT LOGGING

`models.py` — SQLAlchemy Models

Privacy by default: Stores SHA-256 of **normalised prompt**, NOT raw text. Raw storage is opt-in. **AuditLog table:** id, timestamp, prompt_hash, decision, scores, disagreement, escalated, rationale, judge_verdict, policy_mode, latency_ms, channel_latency, degraded

`repo.py` — Repository Layer

- Engine management (SQLite for dev, PostgreSQL via Docker)
 - Session management with context manager
 - `insert_decision()` — Writes audit record
 - `recent_decisions()` — Dashboard live feed
 - `metrics_summary()` — Dashboard stats cards
 - `scores_for_playground()` — Threshold playground recomputation
-

📁 EVAL/ — EVALUATION HARNESS

`datasets.py` — Unified Dataset Builder

Schema: `text`, `label` (0/1), `attack_family`, `source`, `split`, `license` **Families:** `none`, `gcg_optimised`, `jailbreak_roleplay`, `injection_direct`, `injection_indirect`, `obfuscation`

Two modes:

- `--synthetic` — Deterministic offline generator (seeded, zero downloads) — **runs in CI**
- Real HF loaders — AdvBench, JBB, In-The-Wild, deepset, BIPiA, Alpaca, Dolly, NotInject, etc.

Critical: Group splitting by (`attack_family`, `source`) — never random! Near-duplicate DAN variants stay on same side of split. Holdout locked until final eval.

Funnel reporting: `total → after quality filter → after dedup → final` — never raw row counts.

metrics.py — Evaluation Metrics

- Detection bundle: ROC-AUC, PR-AUC, accuracy, precision, recall, F1, FPR, FNR
- **Bootstrap CI on AUC** (1000 resamples) — statistical rigour
- Per-family metrics (aggregate hides complementarity!)
- Latency percentiles (p50/p95/p99)
- Cost per 1000 prompts

runner.py — 12-Configuration Ablation Grid

Runs identical harness across all configs:

#	Config	Channels	Fusion	Escalation	Tests
1	channel_a_only	A	single	none	Perplexity baseline
2	channel_b_only	B	single	none	Embedding baseline
3	deberta_guard_only	Guard	single	none	Off-the-shelf baseline
4	prompt_guard_only	Guard	single	none	Off-the-shelf baseline
5	judge_on_everything	Judge	single	none	Accuracy/cost ceiling
6	naive_mean_fusion	A+B	mean	none	Key comparison
7	naive_max_fusion	A+B	max	none	Alternative baseline
8	dgad_no_escalation	A+B	gate	none	Gating alone?
9	dgad_escalate_judge	A+B	gate	judge	Core system
10	dgad_offset_judge	A+B+E	gate	judge	+Representation offset
11	dgad_full	A+B+E	gate	judge+probe	Full system
12	dgad_full_hardenened	A+B+E	gate	judge+probe	+Adversarial loop

Outputs: CSV per config + `ablation.csv` summary + `gate_tradeoff_curve.csv` + `run_meta.json`

`report.py` — Figure Generator

Generates ALL report figures from raw CSVs (never hand-drawn):

- ROC overlay (DGAD vs baselines)
 - Escalation rate vs AUC trade-off (**headline figure**)
 - Per-family detection rate bars (complementarity story)
 - Cost vs Accuracy Pareto frontier
 - Round-over-round hardening plot (adversarial loop)
-



Docker & CI/CD

`docker-compose.yml`

Services:

- `api` — DGAD FastAPI app (port 8000)
- `postgres` — PostgreSQL 16 (port 5432, persistent volume)
- `redis` — Redis 7 (port 6379)

`Dockerfile`

- Python 3.11 slim base
- `uv` for fast, locked installs
- CPU-only PyTorch first (avoids multi-GB CUDA wheel)
- Exposes 8000, runs uvicorn

`.github/workflows/`

- `ci.yml` — Lint (ruff), type-check (mypy), unit+integration tests, build Docker
 - `eval.yml` — Nightly full benchmark on validation split → MLflow + Prometheus
 - `security.yml` — bandit + pip-audit + detect-secrets
-



Dashboard (React + Vite + TypeScript)

ARCHITECTURE

```

dashboard/
├─ package.json          # Dependencies
├─ vite.config.ts        # Dev server + API proxy
├─ tsconfig.json         # TypeScript config
├─ tailwind.config.js    # Tailwind + design tokens
├─ src/
│   ├─ main.tsx          # Entry point
│   ├─ App.tsx           # Main layout + data fetching
│   ├─ api.ts            # Axios + Zod validation
│   ├─ hooks.ts          # useCountUp animation
│   ├─ theme.ts          # MUI theme (Utilitarian style per ADR 0005)
│   ├─ mock.ts           # Demo data fallback
│   └─ components/
│       ├─ StatCards.tsx # Total, block rate, escalation rate, avg
│       └─ DisagreementScatter.tsx # 🌀 HERO VIEW: A vs B scatter with
latency
├─ EscalationFunnel.tsx # Auto-pass/block/escalate breakdown
├─ ThresholdPlayground.tsx # Interactive threshold sliders (viva demo!)
├─ FamilyBreakdown.tsx # Per-attack-family metrics
└─ PromptDetail.tsx    # Drill-down: all scores, judge rationale,
probe traces

```

KEY DASHBOARD FEATURES

Component	What It Shows	Why It Matters
DisagreementScatter	X=Channel A, Y=Channel B, shaded escalation band	Makes the entire thesis visible in one plot
ThresholdPlayground	Sliders for t_low/t_high/t_d → live recompute	Viva demo: move sliders, watch escalation rate change
LiveFeed	Real-time prompt decisions	Shows system working
PromptDetail	Full breakdown per prompt	Forensic analysis

Design System (ADR 0005): Utilitarian — flat surfaces, hairline borders, industrial type (IBM Plex Sans + JetBrains Mono), restrained colour (stone/stone accent).

Tests — Three Tiers

```
tests/
├── unit/                                # Pure logic, no external deps
│   ├── test_gate.py                    # Disagreement gate exhaustive + property-based
│   ├── test_calibration.py             # Calibration correctness
│   ├── test_statistical.py             # Channel A scoring
│   ├── test_semantic.py                # Channel B scoring
│   ├── test_offset.py                  # Channel E scoring
│   ├── test_behavioural.py             # Channel C scoring
│   ├── test_judge.py                  # Judge parsing, canary, caching
│   ├── test_canary_loop.py             # Canary + adversarial loop
│   ├── test_datasets.py                # Data layer: no leakage, schema, splits
│   └── test_metrics.py                 # Metric calculations
├── integration/                        # Full pipeline, real-ish deps
│   └── test_api.py                     # FastAPI endpoints, DB, cache
└── adversarial/                        # Attack regression suite
    └── test_evasion_regression.py      # CI FAILS if caught attack starts passing
```

Run all: `pytest` (unit + integration) | `pytest tests/adversarial/` (regression suite)

Documentation

DOCS/ADR/ — ARCHITECTURE DECISION RECORDS

ADR	Title	Why It Matters
0001	Disagreement Gating Over Score Averaging	Core novelty justification
0002	Calibration Before Gate	Explains why calibration must precede gating
0003	FastAPI Only	Dropped Node/Express — single backend
0004	Offline-First Fallbacks	Graceful degradation when models/services missing
0005	Dashboard Style (Utilitarian)	Design system rationale

DOCS/RUNBOOK.MD

Operational guide: setup, data build, training, eval, deploy, troubleshooting.

DOCS/VIVA_PREP.MD

22 anticipated viva questions with answer frameworks — **every team member must know all.**

Key Concepts You Must Be Able To Explain

1. WHY DISAGREEMENT GATING?

"Prior systems average scores. If $A=0.2$ and $B=0.9$, $\text{average}=0.55$ — the system shrugs. We treat that disagreement as the signal: this prompt is genuinely hard, pull it aside for expensive verification."

2. WHY CALIBRATION BEFORE GATE?

"Raw scores live on different scales (perplexity 340 vs probability 0.71). Without calibration, 'disagreement' is just scale mismatch, not real information."

3. COMPLEMENTARY BLIND SPOTS

"Channel A catches gibberish but misses fluent role-play. Channel B catches role-play but over-defends on trigger words. **This gap is the empirical justification for our entire architecture.**"

4. TWO NOVELTIES

1. **Disagreement Gate** — Disagreement as routing signal, not number to average
2. **Self-Adversarial Loop** — Actively hunts false-agreement inputs, turns them into training data

5. COST ADVANTAGE

"Judge-on-everything: ~\$X/1K prompts. DGAD: ~\$Y/1K prompts (escalation rate ~15%). The escalation-rate-vs-AUC curve quantifies this trade-off."

6. SECURITY OF THE JUDGE

"The judged prompt is hostile input. We wrap it in data delimiters, add a per-call canary to the judge's system prompt, validate structured output, and fail-closed on hijack."

7. NO DATA LEAKAGE

"Splits by (attack_family, source) groups, not random rows. Near-duplicate DAN variants cannot leak across train/test. Holdout locked until final eval."

Recommended Reading (Small Blogs & Papers)

FOR UNDERSTANDING THE CORE IDEAS

Topic	Resource	Why Read It
Windowed Perplexity	Jain et al. “Baseline Defenses” (arXiv:2309.00614)	The paper behind Channel A
Perplexity Detection	Alon & Kamfonas (arXiv:2308.14132)	Original perplexity-as-detector idea
Over-Defense Problem	InjecGuard (arXiv:2410.22770)	Why NotInject matters
Consistency Confound	OpenReview:B6ZrLXou3u	Why behavioural probe is weak alone — WE CITE THIS OURSELVES
DataSentinel	arXiv:2504.11358	Adversarial training for injection — different from our loop
ROD (Rep Offset)	Search “Representation Offset Detection”	White-box inspiration for our Channel E
GCG Attacks	Zou et al. (arXiv:2307.15043)	The optimisation-based attack we target

FOR ENGINEERING CONCEPTS

Concept	Search Terms	What You'll Learn
Platt vs Isotonic Calibration	"sklearn calibration classifiercv"	When to use which
Optuna Threshold Tuning	"optuna constraint optimization"	How we tune t_low/t_high/t_d
SQLAlchemy Async	"sqlalchemy 2.0 async session"	Our repo.py patterns
FastAPI Dependency Injection	"fastapi depends tutorial"	How <code>get_pipeline()</code> works
React Query (TanStack)	"tanstack query tutorial"	Dashboard data fetching
Recharts Scatter	"recharts scatter chart example"	DisagreementScatter implementation
Zod Runtime Validation	"zod parse vs validate"	api.ts schemas

Viva Preparation Checklist

- ☐ Can explain the disagreement gate in **one sentence** (memorise the novelty statement)
- ☐ Can walk through **one request end-to-end** (normalise → A/B/E → calibrate → gate → escalate? → decide → log)
- ☐ Know **which channel catches what** and **which channel misses what** (the table in final_plan.md §2.3)
- ☐ Can explain **why calibration is on validation only** (not train, not test)
- ☐ Can defend **why behavioural probe is weak alone but useful as tie-breaker** (cite Consistency Confound)
- ☐ Can explain **how the adversarial loop preserves attack intent** (judge verification gate)
- ☐ Can describe **the ablation grid** and **which configs are the key comparisons** (6 vs 9, 9 vs 11)

- ☐ Can explain **OpenAI-compatible proxy** — why it matters for adoption
 - ☐ Can discuss **privacy** — we hash prompts, raw storage is opt-in
 - ☐ Can discuss **failure modes** — judge down, false agreement, overfitting to mutators
 - ☐ Have **honest answer** for “what does the ablation show does NOT contribute?”
-

Final Notes for Your Viva

This project is 94% implemented. Every file in `main/` exists and works. The test suite passes. The dashboard runs. The evaluation harness produces all 12 ablation configurations. The adversarial loop runs. The red-team suite exists.

You built this. Every module has an owner and a reviewer. You can defend every line.

The remaining 6% is Phase 15: Write the report, generate the figures, do the dry runs, prepare the viva answers. That’s writing and presenting — the engineering is done.

When they ask “what’s your contribution?” — Point to `gate.py` (disagreement gate) and `adversarial/loop.py` (self-adversarial loop). Everything else is solid engineering that makes those two ideas testable and measurable.

Good luck! You know this system better than anyone. 🚀

Generated for: Rajpreet Singh Khurana (K033), NMIMS Semester 7 Capstone Project: DGAD — Disagreement-Gated Adaptive Detection